LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 5
“REKURSIF”

DISUSUN OLEH:

RAFI RAMADHAN

109082500140

S1 IF-13-02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2025/2026

DASAR TEORI

Rekursif adalah teknik pemrograman di mana suatu subprogram, baik fungsi maupun prosedur, dapat memanggil dirinya sendiri. Teknik ini digunakan untuk memecahkan suatu masalah dengan cara menyelesaikan sub-masalah yang identik dari masalah utamanya. Sebuah algoritma rekursif selalu memiliki dua komponen utama, yaitu base case dan recursive-case. Base-case (basis) merupakan bagian yang berfungsi menghentikan proses rekursif dan menjadi komponen terpenting; mustahil membuat program rekursif tanpa mendefinisikan base-case terlebih dahulu. Sementara itu, recursive-case adalah kondisi di mana pemanggilan terhadap dirinya sendiri terjadi, yang merupakan negasi dari base case. Saat program rekursif dijalankan, terjadi proses forward, yaitu subprogram terus memanggil dirinya sendiri hingga mencapai kondisi base case yang bernilai true. Setelah base case terpenuhi, terjadilah proses backward, yaitu program kembali ke subprogram sebelumnya yang memanggilnya, dan proses ini berlanjut mundur hingga kembali ke pemanggilan awal. Pendekatan rekursif ini sering menjadi alternatif pengganti struktur perulangan, dan setiap algoritma rekursif dipastikan memiliki padanan dalam bentuk iterative.

SOAL LATIHAN

Rekursif

1. **Fibonacci**

Source Code:

```
package main

import "fmt"

func fibonacci(n int) int {
    if n == 0 {
        return 0
    }
    if n == 1 {
        return 1
    }
    return fibonacci(n-1) + fibonacci(n-2)
}

func main() {
    fmt.Println("Deret Fibonacci (suku ke-0 hingga ke-10):")

    fmt.Print("n  | ")
    for i := 0; i <= 10; i++ {
        fmt.Printf("%-4d", i)
    }

    fmt.Println()
    fmt.Print("Sn | ")
    for i := 0; i <= 10; i++ {
        fmt.Printf("%-4d", fibonacci(i))
    }
    fmt.Println()
}
```

Output:

```
PS C:\alpro2> go run soal1.go
Deret Fibonacci (suku ke-0 hingga ke-10):
n  | 0   1   2   3   4   5   6   7   8   9   10
Sn | 0   1   1   2   3   5   8   13  21  34  55
PS C:\alpro2>
```

Deskripsi Program:
Program ini mengimplementasikan fungsi rekursif untuk menghitung deret Fibonacci dengan kondisi basis saat n bernilai 0 atau 1, di mana nilai suku selanjutnya diperoleh dari penjumlahan dua suku sebelumnya. Keluaran ditampilkan secara horizontal menggunakan dua perulangan terpisah dengan format %-4d sehingga nilai n dan Sn tersusun rapi berpasangan. Meskipun rekursif membuat kode lebih ringkas secara

matematis, pendekatannya lebih kompleks dibandingkan loop biasa karena melibatkan pemanggilan fungsi ke dirinya sendiri secara berulang.

2. **Pola bintang**

Source Code:

```
package main

import "fmt"

func cetakBintang(a int, n int) {
    if a > n {
        return
    }
    for i := 0; i < a; i++ {
        fmt.Print("*")
    }
    fmt.Println()
    cetakBintang(a+1, n)
}

func main() {
    var n int
    fmt.Print()
    fmt.Scan(&n)
    cetakBintang(1, n)
}
```

Output:

```
PS C:\alpro2> go run soal2.go
10
*
**
***
****
*****
******
*******
********
*********
**********
```

Deskripsi Program:

Kesimpulan dari kode program tersebut adalah bahwa program ini berfungsi untuk mencetak pola segitiga siku-siku menggunakan karakter bintang (*) berdasarkan jumlah baris yang diinputkan oleh pengguna. Program ini memanfaatkan teknik rekursif melalui fungsi cetakBintang, di mana fungsi tersebut mencetak sejumlah bintang pada satu baris, lalu memanggil dirinya sendiri secara berulang dengan menambah jumlah bintang pada baris berikutnya hingga mencapai batas maksimal yang telah ditentukan.

3. **faktor bilangan dari suatu N**

Source Code:

```
package main
```

```go
import "fmt"

func faktor(n int, i int) {
	if i > n {
		return
	}
	if n%i == 0 {
		fmt.Print(i, " ")
	}
	faktor(n, i+1)
}

func main() {
	var n int
	fmt.Print("Masukkan bilangan N: ")
	fmt.Scan(&n)
	fmt.Print("Faktor dari ", n, ": ")
	faktor(n, 1)
	fmt.Println()
}
```

Output:

```
PS C:\alpro2> go run soal3.go
Masukkan bilangan N: 8
Faktor dari 8: 1 2 4 8
PS C:\alpro2> go run soal3.go
Masukkan bilangan N: 5
Faktor dari 5: 1 5
PS C:\alpro2>
```

Deskripsi Program:

Program ini mengimplementasikan fungsi rekursif faktor(n, i) untuk mencari dan menampilkan semua faktor dari bilangan bulat positif N dengan cara memeriksa satu per satu bilangan dari 1 hingga N, di mana setiap bilangan i yang habis membagi N (ditandai dengan sisa bagi n%i == 0) akan langsung dicetak, kemudian fungsi memanggil dirinya sendiri dengan nilai i yang bertambah satu hingga i melebihi N sebagai kondisi berhentinya. Pendekatan rekursif ini menghasilkan keluaran faktor yang terurut dari terkecil hingga terbesar secara otomatis karena pemeriksaan dimulai dari i=1.

4. Pola Rekursif

Source Code:

```go
package main

import "fmt"

func cetakPola(n int) {
    if n == 1 {
        fmt.Print(n)
```

```
        return
    }
    fmt.Print(n, " ")
    cetakPola(n - 1)
    fmt.Print(" ", n)
}

func main() {
    var n int
    fmt.Scan(&n)
    cetakPola(n)
    fmt.Println()
}
```

Output:

```
PS C:\alpro2> go run soal4.go
5
5 4 3 2 1 2 3 4 5
PS C:\alpro2> 9
9
PS C:\alpro2> go run soal4.go
9
9 8 7 6 5 4 3 2 1 2 3 4 5 6 7 8 9
PS C:\alpro2>
```

Deskripsi Program:

Program di atas bekerja dengan memanfaatkan konsep fungsi yang memanggil dirinya sendiri secara terus-menerus sebelum mencapai kondisi akhir. Saat fungsi dipanggil, ia akan mencetak angka secara menurun mulai dari nilai inputan pertama dan menahannya di memori, lalu memanggil fungsinya lagi dengan nilai yang dikurangi satu sampai menyentuh angka satu. Setelah batas akhir tercapai dan angka satu dicetak, tumpukan memori tadi akan dilepas perlahan sehingga program bisa mencetak kembali angka-angka tersebut secara menaik hingga kembali persis ke nilai awalnya.

5. Deret ganjil

Source Code:

```
package main

import "fmt"

func cetakGanjil(n int) {
    if n > 0 {
        cetakGanjil(n - 1)
        if n%2 != 0 {
            fmt.Print(n, " ")
        }
    }
}

func main() {
```

```
    var n int
    fmt.Scan(&n)
    cetakGanjil(n)
    fmt.Println()
}
```

Output:

```
PS C:\alpro2> go run soal5.go
5
1 3 5
PS C:\alpro2> go run soal5.go
20
1 3 5 7 9 11 13 15 17 19
PS C:\alpro2>
```

Deskripsi Program:

logikanya lumayan simpel, fungsi itu kan bakal manggil dirinya sendiri terus menerus sampai nilainya habis jadi nol. nah pas prosesnya jalan mundur balik ke atas, dia sekalian ngecek tiap angka pakai sisa bagi. kalau angkanya dibagi dua masih ada sisa berarti kan ganjil tuh, nah langsung dicetak aja deh. makanya hasil akhirnya bisa urut rapi dari satu sampai angka maksimal yang kamu masukin tadi. jangan lupa disave dulu ya sebelum di run biar ngga error.

6. **Implementasikan rekursi**

Source Code:

```
package main

import "fmt"

func pangkat(x, y int) int {
    if y == 0 {
        return 1
    }
    return x * pangkat(x, y-1)
}

func main() {
    var x, y int
    fmt.Scan(&x, &y)
    fmt.Println(pangkat(x, y))
}
```

Output:

```
PS C:\alpro2> go run soal6.go
2 2
4
PS C:\alpro2> go run soal6.go
5 3
125
PS C:\alpro2>
```

Deskripsi Program:

Program ini bekerja dengan cara mengalikan bilangan dasar x dengan dirinya sendiri secara berulang, sementara nilai pangkat y akan terus dikurangi satu di setiap pemanggilan fungsinya. Ketika nilai y sudah mencapai batas nol, proses rekursif akan berhenti dan program akan mengembalikan hasil akhir dari total perkalian tersebut.

DAFTAR PUSTAKA

Fakultas Informatika Universitas Telkom. *Modul 5. Rekursif*. Modul Praktikum Algoritma dan Pemrograman 2.